

13: Rust Senior Interview Guide, Part 3: Gotchas, Full Doubly Linked List, Interview Tactics

This is the last of the three Rust guides. It covers the compiler errors and runtime failures that most often slow candidates down, a doubly linked list built with `Rc`, `RefCell`, and `Weak`, and practical advice for a Rust coding interview.

This chapter covers

- Common borrow checker errors, what causes them, and how to fix each one
- `as_ref`, `take`, `unwrap`, `expect`, and `?`
- Deadlocks, async pitfalls, reference cycles, and `RefCell` panics
- A doubly linked list with `Rc<RefCell<Node<T>>>` and `Weak` back pointers
- How to communicate, test, and discuss trade-offs during a phone screen

Contents

1. Common errors and pitfalls	3
1.1 Mutable borrow while an immutable borrow is active	3
1.2 A temporary value dropped while borrowed	3
1.3 Moving out of a borrowed value	3
1.5 Modifying a collection while iterating over it.....	4
1.6 <code>&</code> , <code>as_ref</code> , and <code>take</code>	5
1.7 <code>unwrap</code> , <code>expect</code> , and <code>?</code>	5
1.8 Deadlocks.....	5
1.9 Async pitfalls.....	6
1.10 Reference cycles with <code>Rc</code>	6
1.11 <code>RefCell</code> borrow panics.....	6
2.1 The data structure	8
2.2 The operations	9
2.3 What to leave out in an interview	12
3. The phone screen	13
3.1 Think aloud	13
3.2 Ask clarifying questions.....	13
3.3 Use the compiler and tests	13
3.4 Discuss trade-offs.....	14
3.5 Working in an online editor	14
3.6 Common problems	14
4. How the Rust guides fit together	15

Summary 16

1. Common errors and pitfalls

Each subsection shows code that fails, explains the rule it breaks, and shows a fix. Blocks marked `rust,ignore` do not compile, intentionally.

1.1 Mutable borrow while an immutable borrow is active

```
``rust,ignore let mut values = vec![1, 2, 3]; let first = &values[0];
values.push(4); // ERROR: cannot borrow values` as mutable because it is also borrowed
as immutable println!("{first}");
```

****Why it fails.**** ``first`` points into the vector's buffer. ``push`` may reallocate the buffer, which would leave ``first`` pointing at freed memory. Rust enforces this rule: while a shared reference is in use, the value cannot be modified.

****Fix.**** Copy the value out, or finish using the reference before the mutation:

```
``rust
let mut values = vec![1, 2, 3];
let first_copy = values[0];
values.push(4);
println!("{first_copy} {values:?}");
```

When you genuinely need shared mutable state, `RefCell` (one thread) and `Mutex` (several threads) enforce the same rule at run time instead of compile time.

1.2 A temporary value dropped while borrowed

```
``rust,ignore let name = String::from("A100").as_str(); // ERROR: temporary dropped
```

****Why it fails.**** ``String::from("A100")`` creates a temporary ``String`` that is dropped at the end of the statement. ``as_str()`` borrows from it, so ``name`` would refer to freed memory. The compiler reports error E0716 as soon as ``name`` is used later.

****Fix.**** Bind the owner to a variable, so it lives as long as the borrow:

```
``rust
let owned = String::from("A100");
let name = owned.as_str();
```

1.3 Moving out of a borrowed value

```
``rust,ignore struct Config { name: String } fn bad(config: &Config) -> String
{ config.name // cannot move out of borrowed content }
```

****Why it fails.**** The function has only a shared reference to ``config``, so it cannot take ownership of the ``name`` field. Moving the field out would leave the caller's ``Config`` incomplete. The compiler reports "cannot move out of ``config.name`` which is behind a shared reference."

****Fix.**** Choose one of three options:

- Return a copy: ``config.name.clone()``.
- Return a borrow: change the return type to ``&str`` and return ``&config.name``.
- Take ownership: change the parameter to ``config: Config`` and return ``config.name``.

1.4 Explicit lifetimes

The compiler infers lifetimes in most function signatures. You must write them when a function returns a reference that could come from more than one input:

```
```rust
fn longest<'a>(a: &'a str, b: &'a str) -> &'a str {
 if a.len() >= b.len() { a } else { b }
}
```

The signature says that the returned reference is valid only as long as both inputs are.

Lifetime parameters spread to every type and function that holds such a reference. To keep code simple, especially in an interview:

- Return owned values (`String`, `Vec<T>`) when the cost is acceptable.
- Use `Cow<'a, str>` when a function returns borrowed data in the common case and owned data only sometimes.
- Let structs own their data, for example `struct Client { base_url: String }` rather than `struct Client<'a> { base_url: &'a str }`.

## 1.5 Modifying a collection while iterating over it

In many languages, modifying a collection during iteration causes undefined behavior or an exception. Rust rejects it at compile time:

```
```rust,ignore let mut items = vec![1, 2, 3]; for item in &items { items.push(4); // ERROR:
cannot borrow as mutable while immutable borrow active }
```

****Fix.**** Collect the changes first, then apply them:

```
```rust
let mut items = vec![1, 2, 3];
let to_add: Vec<i32> = items.iter().map(|x| x + 1).collect();
items.extend(to_add);
```

To remove elements that match a condition, use `retain`.

## 1.6 &, as\_ref, and take

Expression	Result
<code>&amp;x</code>	A reference to <code>x</code>
<code>opt.as_ref()</code>	Converts <code>&amp;Option&lt;T&gt;</code> into <code>Option&lt;&amp;T&gt;</code> , so you can inspect the contents without moving them. Result has the same method
<code>opt.take()</code>	Returns the value and leaves <code>None</code> in its place. Requires <code>&amp;mut</code> access

```
let mut opt = Some(String::from("gpu"));
let borrowed: Option<&String> = opt.as_ref();
let owned: String = opt.take().unwrap();
```

`take` is essential for linked structures: it lets you move a node out of a field that you can access only through a mutable reference (section 2).

## 1.7 unwrap, expect, and ?

Method	Behavior	Use
<code>unwrap()</code>	Panics on <code>None</code> or <code>Err</code>	Tests, examples, and quick prototypes
<code>expect("message")</code>	Panics with your message	When failure would mean a bug, and the message states the invariant
<code>?</code>	Returns the error to the caller	Production functions that can fail

```
fn read_first_line(path: &str) -> std::io::Result<String> {
 let content = std::fs::read_to_string(path)?;
 Ok(content.lines().next().unwrap_or_default().to_string())
}
```

`unwrap_or_default()` returns the default value, here an empty string, instead of panicking when the file has no lines.

## 1.8 Deadlocks

Two patterns cause most deadlocks:

- **Inconsistent lock order.** One thread locks A and then B, while another locks B and then A. Each waits for the lock the other holds.
- **Locking the same mutex twice.** `std::sync::Mutex` is not reentrant, so a thread that calls `lock()` while it already holds the guard waits forever. This often happens when a method that holds a lock calls another method that takes the same lock.

To prevent them:

- **Define a global lock order** and always acquire locks in that order.
- **Hold locks briefly.** Copy out the data you need and drop the guard.
- **Do not call unknown code, such as callbacks, while holding a lock.**
- **Use** `try_lock()` **and** retry with backoff where waiting indefinitely is not acceptable.
- **In async code, do not hold a** `std::sync::MutexGuard` **across** `.await`. If the data can be copied out, use `std::sync::Mutex` and release the guard before awaiting. If the lock must be held across an `await`, use `tokio::sync::Mutex`.

## 1.9 Async pitfalls

- **Blocking in an async task** stops other tasks on the same runtime thread. Use `tokio::task::spawn_blocking` for blocking I/O and CPU-heavy work.
- `std::sync::MutexGuard` **is not** `Send`, so a future that holds one across `.await` cannot be passed to `tokio::spawn`.
- **Any non-`Send` value held across** `.await`, such as an `Rc`, has the same effect.

## 1.10 Reference cycles with `Rc`

If two `Rc` values point to each other, directly or through a chain, their reference counts never reach zero, and the memory is never freed. Rust's safety guarantees do not cover memory leaks. Use `Weak` for references that point back toward an owner, or store nodes in a `Vec` (an arena) and link them with indexes instead of pointers.

## 1.11 `RefCell` borrow panics

```
rust,ignore let cell = std::cell::RefCell::new(0); let a = cell.borrow(); let b =
cell.borrow_mut(); // panics: already borrowed
```

This code compiles, but it panics at run time: `RefCell` allows either many shared borrows or one mutable borrow, and `a` is still alive when `borrow_mut` is called.

To avoid the panic:

- Drop a `Ref` before calling `borrow_mut` on the same cell, for example by putting the borrow in its own block.
- Keep borrows as short as possible, and do not store `Ref` or `RefMut` guards in variables that outlive a single operation.
- Use `try_borrow` or `try_borrow_mut` to receive a `Result` instead of a panic.

---

## ## 2. A doubly linked list

A doubly linked list is a common interview exercise in Rust, because each node is referenced from two directions, and Rust's ownership rules require a single owner. The repository contains a complete implementation with tests:

```
```text
rust-interview-lab/src/bin/ll.rs
```

It provides:

- `new`, `len`, and `is_empty`
- `push_front`, `push_back`, `pop_front`, and `pop_back`
- `peek_front` and `peek_back`
- Extraction of the value from a removed node with `Rc::try_unwrap`
- Weak back pointers, so the nodes do not form reference cycles
- Unit tests for the list's invariants

Run it and its tests:

```
cd rust-interview-lab
cargo run --bin ll
cargo test --bin ll
```

2.1 The data structure

```
use std::cell::RefCell;
use std::rc::{Rc, Weak};

type NodeRef<T> = Rc<RefCell<Node<T>>>;
type WeakNodeRef<T> = Weak<RefCell<Node<T>>>;

struct Node<T> {
    data: T,
    next: Option<NodeRef<T>>,
    prev: Option<WeakNodeRef<T>>, // weak to avoid cycles
}

pub struct LinkedList<T> {
    head: Option<NodeRef<T>>,
    tail: Option<NodeRef<T>>,
    len: usize,
}
```

Each type in the definition has a purpose:

- `Rc` allows a node to have more than one owner. The second node, for example, is owned by the first node's `next` field, and the last node is also owned by the list's `tail` field.
- `RefCell` allows a node to be modified through an `Rc`, which by itself provides only shared references.
- `Weak` **for** `prev` prevents reference cycles. Suppose `prev` were an `Rc`. The first node would own the second through `next`, and the second would own the first through `prev`. Each node would keep the other's reference count above zero, so neither would be freed when the list is dropped. With `prev` as a `Weak`, each pair of adjacent nodes has only one strong reference between them, from the earlier node to the later one, and dropping the list frees every node.

2.2 The operations

```

impl<T> LinkedList<T> {
    pub fn new() -> Self {
        LinkedList { head: None, tail: None, len: 0 }
    }

    pub fn len(&self) -> usize { self.len }
    pub fn is_empty(&self) -> bool { self.len == 0 }

    pub fn push_front(&mut self, data: T) {
        let new_node = Rc::new(RefCell::new(Node {
            data,
            next: None,
            prev: None,
        }));

        match self.head.take() {
            Some(old_head) => {
                old_head.borrow_mut().prev = Some(Rc::downgrade(&new_node));
                new_node.borrow_mut().next = Some(old_head);
            }
            None => {
                self.tail = Some(new_node.clone());
            }
        }
        self.head = Some(new_node);
        self.len += 1;
    }

    pub fn push_back(&mut self, data: T) {
        let new_node = Rc::new(RefCell::new(Node {
            data,
            next: None,
            prev: self.tail.as_ref().map(Rc::downgrade),
        }));

        match self.tail.take() {
            Some(old_tail) => {
                old_tail.borrow_mut().next = Some(new_node.clone());
            }
            None => {
                self.head = Some(new_node.clone());
            }
        }
        self.tail = Some(new_node);
        self.len += 1;
    }

    pub fn pop_front(&mut self) -> Option<T> {
        self.head.take().and_then(|old_head| {
            let next = old_head.borrow_mut().next.take();
            match next {
                Some(next_node) => {
                    next_node.borrow_mut().prev = None;
                    self.head = Some(next_node);
                }
                None => {
                    self.tail = None;
                }
            }
        })
    }
}

```

```

    }
    self.len -= 1;
    let node = Rc::try_unwrap(old_head).ok().unwrap().into_inner();
    Some(node.data)
  })
}

pub fn pop_back(&mut self) -> Option<T> {
  self.tail.take().and_then(|old_tail| {
    let prev = old_tail
      .borrow_mut()
      .prev
      .take()
      .and_then(|weak| weak.upgrade());
    match prev {
      Some(prev_node) => {
        prev_node.borrow_mut().next = None;
        self.tail = Some(prev_node);
      }
      None => {
        self.head = None;
      }
    }
  })
  self.len -= 1;
  let node = Rc::try_unwrap(old_tail).ok().unwrap().into_inner();
  Some(node.data)
}

pub fn peek_front(&self) -> Option<std::cell::Ref<'_, T>> {
  self.head.as_ref().map(|node| {
    std::cell::Ref::map(node.borrow(), |node| &node.data)
  })
}

pub fn peek_back(&self) -> Option<std::cell::Ref<'_, T>> {
  self.tail.as_ref().map(|node| {
    std::cell::Ref::map(node.borrow(), |node| &node.data)
  })
}
}

```

The details to explain in an interview:

- **take() moves nodes out of fields.** `self.head.take()` returns the current head and leaves `None`, so the function owns the old head and can relink the list.
- **push_front on an empty list sets both ends.** The new node becomes the head and the tail, so the list stores two strong references to it.
- **pop_front must clear every strong reference before unwrapping.** When the list has one node, the `None` arm sets `self.tail = None`, which removes the tail's strong reference. When the list has more nodes, the next node's `prev` is a `Weak`, and it is cleared anyway. After that, `old_head` is the only strong reference, so `Rc::try_unwrap` succeeds and `into_inner` moves the data out of the `RefCell`. If any other strong

reference remained, `try_unwrap` would return `Err` and `.unwrap()` would panic, so the `unwrap` doubles as a check of the invariant.

- `pop_back` **upgrades the Weak pointer** to reach the previous node. `upgrade()` returns `None` if the node has already been freed, which cannot happen while the list is consistent.
- `peek_front` **returns** `Ref<'_, T>`, **not** `&T`. A reference into a `RefCell` must keep the runtime borrow active, so the method returns a guard. `Ref::map` narrows the guard from the whole node to its data field.

2.3 What to leave out in an interview

Iterators (`iter`, `iter_mut`, `IntoIterator`, `DoubleEndedIterator`) and a custom `Drop` add a large amount of code to this list without testing anything new. Focus on:

- Ownership with `Rc<RefCell<Node<T>>>`
- Weak back pointers to prevent cycles
- Correct relinking with `Option::take()`
- Extracting values with `Rc::try_unwrap`

If the interviewer asks about iteration, explain the trade-off:

"With `RefCell`, I cannot return normal `&T` references because borrows are runtime-checked. I could return `Ref<'_, T>`, or I could provide an owned iterator that pops from the list, but that is more machinery than this question needs."

You can also mention that production Rust code seldom uses this design. An arena, a `Vec` of nodes linked by index, avoids `Rc` and `RefCell` entirely and is faster; the LRU cache in `rust-interview-lab/src/problems/lru_cache_easy.rs` uses that approach.

A long list has one more issue: dropping it recursively drops each next field inside the previous node's destructor, which can overflow the stack for millions of nodes. A `Drop` implementation that pops nodes in a loop avoids the recursion.

3. The phone screen

3.1 Think aloud

Explain your reasoning as you work:

1. Restate the problem and confirm the constraints with examples.
2. Describe your approach and its time and space complexity before you write code.
3. Say what you are about to write before you write each part.

For example:

"I'll use a VecDeque for BFS because I need $O(1)$ pop from the front and push to the back. I'll track visited in a HashSet. Time is $O(n)$, space is $O(n)$."

3.2 Ask clarifying questions

- What inputs are valid? Can the input be empty? Can numbers be negative? How large can the input be?
- Must the output preserve the input order?
- Will the code run on one thread or several?
- Is the caller another library or a person using a CLI?

3.3 Use the compiler and tests

- Write the function signature first. It fixes the ownership decisions: whether the function borrows or takes ownership, and what it returns.
- Write a few `assert_eq!` tests, before or right after the implementation.
- When the borrow checker reports an error, read the full message and explain the fix aloud. Interviewers want to see that you understand the error, not only that you resolve it.
- Run `cargo test` if you have a local environment. In an online editor, write a small `#[cfg(test)]` module and run it.

3.4 Discuss trade-offs

Choice	Trade-off
Vec or LinkedList	Cache locality almost always favors Vec
HashMap or BTreeMap	Expected $O(1)$ lookup, or ordering and range queries
Mutex or RwLock	Depends on the ratio of reads to writes and the length of critical sections
Rc<RefCell<T>> or an arena with indexes	Flexible pointers with runtime checks, or faster code with manual index management
Blocking or async request	Simpler code, or many concurrent requests on few threads

3.5 Working in an online editor

- Use autocomplete to confirm method names instead of guessing.
- Let compiler errors guide each change.
- Keep functions small so each one can be tested on its own.
- Prefer clear code to clever one-liners, unless the one-liner is the common idiom, such as `*map.entry(k).or_insert(0) += 1`.

3.6 Common problems

Problem	Approach	Implementation in rust-interview-lab
Two Sum	HashMap from value to index	src/problems/two_sum.rs
Valid parentheses	Vec as a stack	src/problems/valid_parentheses.rs
LRU cache	HashMap plus a linked structure or an index-linked arena	src/problems/lru_cache.rs, src/problems/lru_cache_easy.rs
Rate limiter	Token bucket, or a deque of timestamps	src/problems/rate_limiter.rs
Worker pool	mpsc channel with Arc<Mutex<Receiver>>	src/problems/worker_pool.rs
Number of islands	BFS or DFS over a grid	src/bin/count_islands.rs
Merge intervals	Sort, then match on last_mut()	src/problems/merge_intervals.rs
Autocomplete	Trie	src/problems/trie.rs
Dependency ordering	Topological sort	src/problems/graph_topology.rs
Top-k	BinaryHeap	src/problems/top_k_frequent.rs

4. How the Rust guides fit together

1. **Part 1** (11-rust-guide-part1-collections-adt-smart-pointers.md) covers collections, API conventions, algebraic data types, and smart pointers.
2. **Part 2** (12-rust-guide-part2-errors-concurrency-sdk.md) covers errors, concurrency, async, and SDK and CLI design.
3. **Part 3**, this guide, covers common errors, the linked list, and interview technique.
4. **Runnable examples** are in rust-interview-lab:

```
cd rust-interview-lab
cargo test
cargo run --bin ll
cargo run --bin count_islands
cargo run --bin mutex_poisoning
cargo run --bin singly_linked_list
cargo run --bin syscall_overhead
cargo run --bin shadowing
```

1. **The role and study plan** are in 01-role-map-and-study-plan.md, and the operator and GPU guides are 04, 05, and 10.

Summary

- Most borrow checker errors come from one rule: a value cannot be modified while a shared reference to it is in use. Copy, reorder, or restructure ownership to fix them.
- Use `take()` to move values out of fields, `as_ref()` to inspect without moving, and `?` to propagate errors in production code.
- Prevent deadlocks with a consistent lock order and short critical sections, and do not hold a standard mutex guard across `.await`.
- A doubly linked list in safe Rust uses `Rc<RefCell<Node<T>>>` forward and `Weak` backward; `Rc::try_unwrap` extracts values once all strong references are cleared.
- In the interview, clarify constraints, state complexity before coding, test with `assert_eq!`, and explain trade-offs.

Source: [13-rust-guide-part3-gotchas-linkedlist-interview.md](#) · [PDF](#) · [All study guides](#)